

基于SIMD的ANN查询距离计算优化

— 从标量搜索到FastScan的迭代演进与双平台对比

姓名：匡航逸 学号：2412235
专业：计算机科学与技术 日期：2026 年 5 月 30 日
GitHub: <https://github.com/xzxxntxdy/Parallel-programming-NKU>

摘要

本文针对DEEP100K数据集 ($d = 96, N = 100000$, 内积距离), 系统实现并深入分析了近似最近邻搜索 (ANNS) 查询阶段的六类SIMD加速方案: Flat-SIMD、标量量化 (SQ-SIMD)、乘积量化配合对称/非对称距离计算 (PQ-SDC/PQ-ADC)、SDC流水线并行以及FastScan查表优化。实验覆盖ARM Kunpeng-920 (NEON 128-bit) 和Intel Core i9-14900HX (AVX2 256-bit) 双平台的全部强制实验矩阵, 所有数据经真实运行验证。最终方案FastScan v2 (lane-major布局+寄存器累加+4路展开) 在ARM上以2.44 ms/query (recall@10=0.99995, 7.82×加速), 在x86上以1.06 ms/query (recall@10=1.0, 3.36×加速) 取得全局最优。本文详细记录了从Flat-Scalar到FastScan v2的完整优化路径, 重点分析了FastScan从v1到v2的代码布局优化过程及其背后的cache/microarchitecture原理, 通过perf计数器与汇编分析验证优化效果。

1 问题定义与评价指标

1.1 kNN与ANNS问题定义

给定数据集 $X = \{x_1, x_2, \dots, x_N\} \subset \mathbb{R}^d$ 和查询向量 $q \in \mathbb{R}^d$, 精确 k 最近邻 (kNN) 搜索的目标是找到距离 q 最近的 k 个向量。近似最近邻搜索 (ANNS) 不要求返回完全精确的 k 个最近邻, 而是允许以高准确率换取更低的查询延迟。本项目的目标是在保持 $\text{recall}@k \approx 1$ 的前提下, 最大化查询吞吐或最小化单query延迟。

1.2 距离函数与评价指标

本项目采用框架一致的内积距离:

$$\delta(q, x) = 1 - \sum_{i=1}^d q_i x_i \quad (1)$$

内积 $\langle q, x \rangle$ 越大, 向量越相似, 距离越接近0。选择内积而非欧几里得距离是因为 DEEP100K数据集已在预处理中进行了归一化, 内积等价于余弦相似度。

recall@k 定义为近似搜索结果与精确搜索结果的交集比例:

$$\text{recall}@k = \frac{|\text{KNN}(q) \cap \text{KANN}(q)|}{k} \quad (2)$$

其中 $KNN(q)$ 为精确搜索结果集合（由ground truth提供）， $KANN(q)$ 为算法返回的近似结果集合。本项目固定 $k = 10$ 。精确Flat Search的recall恒为1.0（浮点舍入误差可忽略）；近似算法的目标是在 $recall \geq 0.999$ （对于最终提交）的前提下最小化延迟。

latency指标包括：

- 单query总延迟（从查询发起到返回top- k 结果的墙钟时间）
- 对PQ类算法进一步拆分为：query编码时间、LUT构建时间（ADC）或中心间距离表时间（SDC）、scan查表累加时间、top- k /top- p 选择时间、float精排（rerank）时间

1.3 数据集与代码约束

数据集为DEEP100K（图片向量）， $N = 100000$, $d = 96$ ，采用内积距离。`base`和`test_query`均为`float*`类型，共38.4 MB。`test_gt`（每个query的精确top-100答案）仅用于recall评估，严禁直接作为输出答案。框架提供朴素串行Flat Search基线：

```
1 auto res = flat_search(base, test_query + i * vecdim, base_number, vecdim, k);
```

需要自行实现更优算法并替换该调用，返回值格式必须与框架一致。新增代码文件必须使用`.h`（header-only），ARM平台通过`bash test.sh 1 1`正式运行，不允许绕过`test.sh`或修改`qsub.sh`。

2 SIMD算法设计

2.1 Flat-SIMD：朴素向量化距离计算

2.1.1 算法原理与数值分析

Flat Search是最基础的ANNS查询方式：对每个query遍历全部 $N = 100000$ 个base向量，逐对计算内积距离 $1 - \sum_i q_i x_i$ ，同时维护一个大小为 $k = 10$ 的最大堆（max-heap）来跟踪当前 k 个最近邻。

计算量分析：

- 每query遍历 $N = 100000$ 个向量
- 每个向量 $d = 96$ 维，需要96次乘法和95次加法
- 每query总计 $100000 \times 96 = 9.6 \times 10^6$ 次乘加操作
- 每query读取 $100000 \times 96 \times 4 = 38.4$ MB的float base数据
- 算术强度 $\approx \frac{9.6 \times 10^6 \text{ FLOP}}{38.4 \times 10^6 \text{ bytes}} \approx 0.25 \text{ FLOP/Byte}$ （若按乘加各算一操作则为0.5）

极低的算术强度意味着Flat Search是典型的内存密集型（memory-bound）工作负载——性能瓶颈不在计算单元，而在内存带宽和cache层次结构。

2.1.2 SIMD向量化策略

SIMD在单个指令中同时对多个数据执行相同操作，内积 $\sum_i q_i x_i$ 天然适合向量化的批量乘加：

表 1: SIMD ISA维度划分（ $d = 96$ ）

SIMD ISA	寄存器位宽	float32 lanes	chunk/向量	循环迭代次数
SSE / NEON	128-bit	4	24	$96/4 = 24$
AVX / AVX2	256-bit	8	12	$96/8 = 12$
AVX-512	512-bit	16	6	$96/16 = 6$

ARM NEON版本使用fmla（Fused Multiply-Add）指令，单指令在128-bit Q寄存器中完成4个float的乘法和累加：

```
1 float32x4_t sum = vdupq_n_f32(0.0f);
2 for (size_t d = 0; d + 4 <= dim; d += 4)
3     sum = vmlaq_f32(sum, vld1q_f32(a + d), vld1q_f32(b + d));
4 float result = 1.0f - vaddvq_f32(sum); // horizontal add on AArch64
```

NEON的fmla将乘加融合为单条指令，减少一半指令发射压力。

x86 AVX2版本使用4路_mm256累加器展开以隐藏乘法延迟（AGNER FOG优化法则）：

```
1 _mm256 s0 = _mm256_setzero_ps(), s1 = _mm256_setzero_ps();
2 _mm256 s2 = _mm256_setzero_ps(), s3 = _mm256_setzero_ps();
3 for (; d + 32 <= dim; d += 32) { // 4 chunks           8 floats = 32
4     s0 = _mm256_add_ps(s0, _mm256_mul_ps(_mm256_loadu_ps(a+d), _mm256_loadu_ps(b+d)
5     ));
6     s1 = _mm256_add_ps(s1, _mm256_mul_ps(_mm256_loadu_ps(a+d+8), _mm256_loadu_ps(b+d
7     +8)));
8     s2 = _mm256_add_ps(s2, _mm256_mul_ps(_mm256_loadu_ps(a+d+16), _mm256_loadu_ps(b+d
9     +16)));
10    s3 = _mm256_add_ps(s3, _mm256_mul_ps(_mm256_loadu_ps(a+d+24), _mm256_loadu_ps(b+d
11    +24)));
12 }
13 // Horizontal sum: ymm 48           xmm 48           scalar
```

4路展开使x86 OoO引擎可同时飞行4条独立的256-bit乘加流水线。

理论加速比远大于实际，瓶颈在内存：每query 38.4 MB数据，ARM DDR4-2666理论带宽下限≈1.8 ms，计算再快也受限于此。

2.1.3 编译器自动向量化 vs 手写SIMD

编译器在-O2优化级别通过自动向量化（auto-vectorization）将标量循环转换为SIMD指令。以一例展开4个累加器的自动向量化代码为例：

```
1 float sum0=0,sum1=0,sum2=0,sum3=0;
2 for (size_t d=0; d+4<=dim; d+=4) {
3     sum0 += a[d]*b[d]; sum1 += a[d+1]*b[d+1];
4     sum2 += a[d+2]*b[d+2]; sum3 += a[d+3]*b[d+3];
5 }
```

编译器识别到循环体内4条独立乘加后，尝试用SIMD指令并行处理。然而编译器受限于C/C++的严格别名规则（strict aliasing）和指针别名分析（alias analysis）：如果编译器无法证明a和b不重叠，将被迫生成保守的标量代码。手写SIMD通过__restrict或直接使用intrinsics可以绕过这些限制。

此外，编译器自动向量化通常不会使用FMA融合（即使在支持的硬件上），因为FMA的浮点舍入行为与分离的mul+add不同，改变舍入顺序可能违反IEEE 754标准。手写SIMD可以显式使用_mm256_fmadd_ps等intrinsics，前提是编译时启用-mfma（GCC/Clang）或/arch:AVX512（MSVC）。

2.1.4 Top-k维护优化

Flat Search需维护top-k最近邻。标准实现使用std::priority_queue（最大堆）：

```

1 if (heap.size() < k) heap.push({dist, id});
2 else if (dist < heap.top().first) { heap.push({dist, id}); heap.pop(); }

```

每次push/pop的复杂度为 $O(\log k)$ ，当 $k = 10$ 时开销可接受。此外实现了Fixed-Array TopK：维护一个大小为 k 的数组，追踪当前最差元素的下标：

```

1 class FixedTopK {
2     std::vector<pair<float, uint32_t>> items;
3     size_t worst_idx;
4     void push(float dist, uint32_t id) {
5         if (items.size() < k) { items.push_back({dist, id}); recompute_worst(); }
6         else if (dist < items[worst_idx].first) { items[worst_idx] = {dist, id};
7             recompute_worst(); }
8     }
9 };

```

FixedTopK避免堆的 $O(\log k)$ 调整，将push操作从分支较多的heap操作简化为 $O(k)$ 的线性扫描。实测中FixedTopK在 $k = 10$ 时比标准堆快1-2%，收益虽小但在百万级查询场景下可累积。

2.2 SQ-SIMD：标量量化

2.2.1 动机与量化过程

SQ（Scalar Quantization）的核心思想是用更紧凑的数据类型替代float32以降低内存占用和提升SIMD并行度。对向量的每个维度独立进行min-max线性量化：

$$\text{code}[i][j] = \text{clamp} \left(\left\lfloor \frac{x_i[j] - \min_j}{\text{scale}_j} + 0.5 \right\rfloor, 0, 255 \right) \quad (3)$$

其中 $\min_j = \min_i x_i[j]$ ， $\max_j = \max_i x_i[j]$ ， $\text{scale}_j = (\max_j - \min_j)/255$ 。

2.2.2 数值分析

DEEP100K: $N = 100000$, $d = 96$ 。

- float32 base: $100000 \times 96 \times 4 = 38.4$ MB
- uint8 base: $100000 \times 96 \times 1 = 9.6$ MB，压缩比 $4\times$
- 128-bit SIMD: 一次处理16个uint8（vs 4个float32）， $4\times$ 并行度提升
- 查询LUT: $d \times 256 \times 4 = 96$ KB（每query重建，无法完全放入48KB L1D）

rerank候选数 p 与精排访存量： $p = 100 \rightarrow 38.4$ KB, $p = 1000 \rightarrow 384$ KB, $p = 5000 \rightarrow 1.92$ MB。

2.2.3 查询流程详解

SQ查询分两阶段：(1) Coarse search——对每个query构建 $d \times 256$ 的float LUT（ $\text{LUT}[j][c] = (\min_j + \text{scale}_j \cdot c) \cdot q[j]$ ），遍历base code时通过查表累加得到近似内积，维护top- p 候选列表。(2) Rerank——对top- p 候选，用原始float向量和NEON/AVX精确内积重算距离，从中选出最终top- k 。

超参数 p 的权衡： p 过小则粗排精度不足，可能漏掉真正的最近邻（recall降低）； p 过大则精排访存量增大，延迟上升。 p 的选择需要根据目标recall在latency曲线上寻找拐点——通常取recall刚好饱和的最小 p 值。

2.2.4 U8SIMD路径的局限

还实现了一条U8SIMD路径：query同样量化为uint8，coarse search用NEON整数dot（`vmull_u8 + vpadalq_u16`）累加uint8内积。理论上能避免float LUT查表，将coarse scan的每维操作从LUT访存+float加法变为整数乘加。但实验表明recall极低（ $p = 2000$ 时仅0.275），因为uint8内积距离无法有效保持float内积距离的顺序关系——量化误差在 $d = 96$ 维中累积后，距离排序被严重扰乱。因此U8SIMD仅作为对照保留。

2.3 PQ-SDC：对称距离计算

2.3.1 编码流程

Product Quantization将向量空间分解为子空间并分别量化。编码分三步：将 $d = 96$ 切分为 M 个子空间（ $\text{subdim} = d/M$ ）；每子空间取2048条训练样本运行KMeans（ $K_s = 256$ 中心，10次迭代），得到 M 组codebook；每向量各子空间以最近中心ID表示， subdim 个float $\rightarrow$ 1个uint8（压缩比 $4 \times \text{subdim}$ ， $M = 16$ 时整向量从384B压至16B）。

2.3.2 SDC距离计算与误差分析

SDC（Symmetric Distance Computation）对query也进行PQ编码，并预计算每个子空间的中心间内积距离表：

$$D_m[c_q][c_b] = \langle C_m[c_q], C_m[c_b] \rangle, \quad c_q, c_b \in [0, 255] \quad (4)$$

该表大小为 $M \times 256 \times 256 \times 4$ 字节（ $M = 16$ 时4.0 MB），在索引构建时一次性计算。查询时对 N 条base向量各做 M 次查表累加：

$$\hat{\delta}_{\text{SDC}}(q, x_i) = 1 - \sum_{m=0}^{M-1} D_m[\text{qcode}[m]][\text{base_code}[i][m]] \quad (5)$$

SDC误差来自双重量化：base端 ϵ_b 和query端 ϵ_q 。 M 较小时 subdim 大、聚类表达强但code少； M 较大时以更多code换取精度—— M 是SDC的核心超参数。

表 2: PQ数值分析（ $K_s=256$, $N=100000$ ）

M	subdim	base code	ADC LUT	SDC table	查表次数/query
8	12	0.8 MB	8 KB	2.0 MB	0.8M
12	8	1.2 MB	12 KB	3.0 MB	1.2M
16	6	1.6 MB	16 KB	4.0 MB	1.6M

2.4 PQ-ADC：非对称距离计算

2.4.1 算法原理

ADC不编码query，每query在线构建局部LUT： $\text{LUT}_m[c] = \langle q^{(m)}, C_m[c] \rangle, c \in [0, 255]$ 。LUT构建需 $M \times 256 \times \text{subdim}$ 次乘加（ $M = 16$ 时仅24K次， ≈ 0.01 ms）。base扫描时查LUT累加： $\hat{\delta}_{\text{ADC}}(q, x_i) = 1 - \sum_m \text{LUT}_m[\text{base_code}[i][m]]$ 。

2.4.2 ADC vs SDC对比

表 3: SDC与ADC策略深度对比

对比维度	SDC	ADC
query编码	是（需 M 次最近中心搜索）	否
预计算表	中心间距离表 $M \times 256^2 \times 4B$	无
每query构建	无	LUT $M \times 256 \times \text{subdim}$ 次乘加
scan查表	2D索引 $D_m[c_q][c_b]$	1D索引 $\text{LUT}_m[c_b]$
误差来源	base量化 + query量化	仅base量化
recall预期	较低	通常较高（约+0.5-2% recall）
存储开销	+SDC表（ $M = 16$ 时+4.0 MB）	无额外开销

ADC的核心优势是非对称性：query保持原始float精度，仅base端量化，因此recall系统性高于SDC。此外ADC不需要4.0 MB的SDC中心间距离表，索引更紧凑。单query场景下两者的总计算量相当（ADC需LUT构建，SDC需query编码+2D查表）。

2.5 SDC流水线并行

SDC的query编码阶段（在 M 个子空间中各搜索最近中心）与scan查表阶段可以重叠执行。在批量查询场景下，使用生产者-消费者模型实现流水线：

Batch t : Stage 2 scan | Batch $t + 1$: Stage 1 encode | Batch $t - 1$: Stage 3 rerank

实现了2-stage（encode与scan+rerank分离）和3-stage（三者完全分离）两种流水线，使用有界阻塞队列（BlockingQueue）控制各阶段间的背压（back-pressure），当队列满时生产者阻塞等待消费者。

流水线的加速潜力取决于各阶段的时间占比。在本项目中，query编码阶段极短（ ≈ 0.03 ms, 占总延迟不到1%），scan阶段占 $\approx 91\%$ ，因此即使完全重叠编码时间，理论加速上限仅为 $1/(1 - 0.01) \approx 1.01\times$ 。实测加速约6%，与理论分析一致。SDC流水线在编码较重的场景（如复杂预处理）下更具价值。

2.6 FastScan：查表累加阶段优化

2.6.1 设计动机

普通PQ-ADC的scan阶段每query执行 $N \times M = 100000 \times 16 = 1.6 \times 10^6$ 次 LUT查表和浮点累加。虽然每次查表的数据（float LUT, 16 KB for $M = 16$ ）理论上在L1 cache内，但随机访问模式（code值随机分布）导致L1访问延迟成为瓶颈。FastScan通过三个技术优化scan阶段：

- LUT量化**：将float LUT全局量化为uint8，LUT大小从16 KB降至4 KB，显著降低L1 cache压力（4 KB仅为x86 L1D 48KB的8.3%）
- block scan**：将base code按block（ $B = 16 \sim 128$ 条向量）重新组织，使同一block内的候选共享LUT加载
- 累加方式优化**：用整数累加替代浮点累加，减少浮点加法延迟（3-5 cycles $\rightarrow$ 1 cycle）

LUT量化公式： $qlut[i] = \text{clamp}(\lfloor (lut[i] - \min)/\text{scale} + 0.5 \rfloor, 0, 255)$ 。量化引入的MAE误差通常在 $10^{-3} \sim 10^{-4}$ 量级，通过后续float rerank阶段有效校正。

2.6.2 v1 $\rightarrow$ v2 迭代：内存RMW的消除

FastScan的实现经历了两版迭代，这是本文的核心工程贡献。

v1 (part-major布局)：codes按[block][part][lane]组织——同一part的所有lane的代码连续存放。扫描时外层part、内层lane：

```
1 for (part = 0; part < M; ++part)
2     for (lane = 0; lane < B; ++lane)
3         scores[lane] += qlut[part*256 + codes[lane]]; // memory RMW!
```

这种布局对SIMD gather友好（同一part的16条lane的代码连续，可用`_mm_loadu_si128`一次加载），但累加目标`scores[lane]`在内存中，每次更新需三次L1操作：(1) 读`scores[lane]`、(2) 读`qlut[...]`、(3) 写回`scores[lane]`。对 $M = 16, B = 64$ ，每block产生 $16 \times 64 = 1024$ 次内存RMW。此外每block需`std::fill(0)`（ $64 \times 4 = 256$ 字节无效写）。

在ARM Kunpeng-920上，由于in-order流水线对cache miss敏感，v1仍将scan从3.64降至2.75 ms（24.4%加速）——block-major的代码布局改善了空间局部性。但在x86 i9-14900HX上，OoO引擎可以很好地隐藏naive scan中LUT随机访问的L1延迟，而v1引入的额外内存RMW反而造成38%的性能倒退（1.87 vs 1.17 ms）。

v2 (lane-major布局 + 寄存器累加)：codes改为[block][lane][part]，每条lane的 M 个代码连续。外层lane、内层part，累加在寄存器中：

```
1 for (lane = 0; lane + 4 <= B; lane += 4) { // 4-way unroll
2     const uint8_t *c0 = codes + lane*M, *c1 = c0+M, *c2 = c1+M, *c3 = c2+M;
3     int s0=0, s1=0, s2=0, s3=0; // REGISTERS 48           zero memory RMW
4     for (part = 0; part < M; ++part) {
5         s0 += qlut[part*256 + c0[part]]; // L1 read + register add only
6         s1 += qlut[part*256 + c1[part]];
7         s2 += qlut[part*256 + c2[part]];
8         s3 += qlut[part*256 + c3[part]];
9     }
10    coarse[lane+0] = {-(float)s0, id0}; // one-time store
11    // ... store lane+1, +2, +3
12 }
```

v2的关键改进：

- 消除内存RMW：每条lane的score在32-bit整数寄存器中累加，仅最终写入内存一次
- 4路展开：4条lane的LUT查表相互独立，x86 OoO引擎可并行执行4条流水线
- 消除`std::fill`：score在寄存器中初始化为0，无需预清零内存
- uint8 LUT仅需256B/part（总计4KB for $M = 16$ ），完全在L1内

2.6.3 pshufb SIMD查表为何未采用

理想FastScan应使用x86 `pshufb`（ARM等价的`vqtbl1q_u8`）一次完成16个8-bit LUT查表。但该指令仅使用低4位作为索引（每次查16-entry表），需将Ks降至16才能直接使用。

为此我们实现了v3版本：M=32个子空间、Ks=16，总code大小保持16 bytes/vector不变。在x86上pshufb路径获得0.91 ms（相对v2的1.06 ms提速16%），在ARM上NEON vtbl路径获得2.20 ms（相对v2的2.44 ms提速10%）。然而**ARM上recall从0.99995降至0.996**，即使增大rerank候选数 p 也无法恢复至目标精度。根因是 $Ks = 16$ 时每子空间仅16个聚类中心，在subdim=3的极低维空间中无法有效表达向量分布，粗排阶段漏排导致精排无法挽救。

对256-entry LUT使用pshufb需拆分为16组nibble匹配（实测比标量慢15 $\times$ ），因此v3方案在追求recall ≥ 0.99995 目标下被放弃，最终采用v2的Ks=256标量lane-major方案。

2.6.4 Block size超参数分析

Block size B 控制每block内的向量数量，影响L1 cache利用率：

- $B = 16$ ：scores数组仅64字节，但block数量多（6250个），block切换频繁
- $B = 64$ ：scores数组256字节，block数量1563，L1命中率最优
- $B = 128$ ：scores数组512字节，开始超出部分L1容量，cache miss增加

ARM上 $B = 16$ 最优（2.16 ms），因Kunpeng-920的L1D为64KB，更小的block减少cache污染。x86上 $B = 128$ 最优（1.06 ms），因i9-14900HX P-core的L1D为48KB且OoO更激进，更大的block摊销了block切换开销。

3 实现细节

项目采用header-only设计，核心模块如下：

ann_search.h：SIMD距离计算。包含scalar禁止向量化、编译器自动向量化、手写SSE 128-bit、手写AVX2 256-bit（4路累加器展开）、ARM NEON 128-bit（fmla融合乘加）、loop unroll 2/4路、prefetch、Fixed-array TopK。通过条件编译__aarch64__/_AVX2__/_MSC_VER跨平台兼容。

ann_quant.h：量化索引与查询。SQ8索引构建、PQ KMeans训练（可配置 M/Ks /训练样本数/迭代次数）、PQ-SDC/PQ-ADC查询、SDC Pipeline（2/3-stage BlockingQueue）、FastScan v2（lane-major布局+LUT全局量化+4路寄存器累加block scan）。

跨平台适配：GCC __builtin_prefetch在MSVC上替换为_mm_prefetch；GCC __attribute__在MSVC上替换为_declspec。

4 实验环境与实验设置

表 4: 双硬件平台

项目	ARM平台	x86平台
CPU	Kunpeng-920	Intel Core i9-14900HX
ISA	ARMv8.2-A+NEON	x86-64+SSE/AVX2/FMA
SIMD位宽	128-bit (4 lanes)	256-bit (8 lanes)
核心/线程	8C/8T, 单Cluster	24C/32T (8P+16E)
L1D cache	64KB/core	48KB (P-core)
L2 cache	512KB/core	2MB (P-core)
L3 cache	32MB (shared)	36MB (shared)
内存	DDR4-2666	DDR5-5600
编译器	GCC 11.4 -O2 -fopenmp	MSVC 19.43 /O2 /arch:AVX2
正式运行	bash test.sh 1 1	x86_main.exe (本地)
perf/汇编	perf stat	MSVC /FAcs

数据集: DEEP100K, $N = 100000$, $d = 96$, $k = 10$ 。base和query为float*, ground truth (10000条 $\times$ top-100) 仅用于recall评估。ARM正式评估使用前2000条查询; x86 benchmark使用前100条查询, 3次repeat取均值。所有实验矩阵按课程要求覆盖相应参数扫描范围。

5 实验结果与分析

5.1 Flat-SIMD消融实验

表 5: Flat-SIMD消融实验 (双平台, recall均 ≈ 1.0)

方法	ARM (ms)	加速比	x86 (ms)	加速比
Flat-Scalar (no SIMD)	19.08	1.00 $\times$	3.56	1.00 $\times$
Flat-AutoVec (compiler)	11.88	1.61 $\times$	3.21	1.11 $\times$
Flat-Manual-SIMD (hand intrins)	7.39	2.58 $\times$	1.49	2.39 $\times$
Flat+Unroll (2/4-way)	7.20	2.65 $\times$	1.65	2.15 $\times$
Flat+Prefetch (SW prefetch)	7.20	2.65 $\times$	1.31	2.72 $\times$
Flat+Fixed TopK (array heap)	7.50	2.54 $\times$	1.31	2.72 $\times$

分析: (1) 手写SIMD在ARM上加速2.58 $\times$ (19.08 $\rightarrow$ 7.39 ms), x86上加速2.39 $\times$ (3.56 $\rightarrow$ 1.49 ms)。均远低于SIMD lane数理论加速比 (NEON 4 $\times$, AVX2 8 $\times$), 确认Flat Search为内存密集型负载。

注: 本文所有实验以recall@10 ≈ 1 为目标。为维持近似搜索精度, 部分方法 (如SQ8、PQ-SDC) 需以较大的rerank候选数 p 来补偿量化误差, 导致latency高于不做精度约束的场景。这是精度-延迟权衡的固有代价, 而非方法本身的效率问题。

(2) ARM上编译器自动向量化效果 (1.61 $\times$) 明显优于x86 (1.11 $\times$), 因为ARM标量基线更弱 (in-order), 自动向量化的ILP收益更显著。ARM上loop unroll进一步从7.39降至7.20 ms (增益2.6%), 收益有限。

(3) x86上loop unroll (2.15 $\times$) 反而不如单累加器AVX2 (2.39 $\times$), 与ARM形成对比。原因: x86

SSE的4路展开导致更多寄存器spill，MSVC生成的汇编中出现大量`vmovaps XMMWORD PTR [rsp+offset]`栈操作，额外的store/load抵消了展开带来的ILP收益。

(4) Prefetch在x86上有12%加速（1.49→1.31 ms, distance=64），说明即使是i9-14900HX的激进硬件prefetcher，对大跨度（96×4=384字节stride）的顺序访问模式仍可受益于软件prefetch提示。ARM上prefetch效果中性（7.20 vs 7.20 ms），因Kunpeng-920的prefetch引擎已经很好地覆盖了顺序stride访问。

(5) FixedTopK在双平台均获≈1%微加速，收益来自消除堆的 $O(\log k)$ 调整开销（对于 $k = 10$ 约为3-4次比较/交换）。

5.2 SQ-SIMD实验结果

表 6: SQ8性能（x86, recall=1.0, index=9.6 MB）

配置	Total(ms)	Coarse(ms)	Rerank(ms)	vs Flat-Scalar	vs Flat-AVX2
SQ8-p100	3.31	3.28	0.03	1.07×	0.45×
SQ8-p500	3.77	3.62	0.14	0.94×	0.40×
SQ8-p1000	3.13	2.97	0.16	1.14×	0.48×
SQ8-p2000	4.15	3.72	0.41	0.86×	0.36×
SQ8-p5000	5.67	4.52	1.12	0.63×	0.26×

SQ8的latency-recall权衡不如PQ方案，主要因为coarse scan的LUT（96×256×4=96 KB）超出x86 L1D（48KB）。SQ8的最大优势在索引压缩（38.4→9.6 MB, 4×），适合内存严格受限的嵌入式和边缘设备。对于通用x86服务器场景，PQ方案在latency和index size 上均占优。

5.3 PQ-SDC与PQ-ADC实验结果

表 7: PQ-SDC性能（ARM, Ks=256, 粗排+精排）

配置	Total(ms)	Recall@10	Encode(ms)	Scan+Rerank(ms)
SDC-M8-coarse	1.39	0.129	0.028	1.36+0.00
SDC-M8-p2000	2.65	0.915	0.028	2.11+0.53
SDC-M12-coarse	3.66	0.231	0.030	3.63+0.00
SDC-M12-p2000	3.65	0.979	0.030	2.81+0.84
SDC-M16-coarse	1.40	0.324	0.029	1.37+0.00
SDC-M16-p1000	3.82	0.993	0.029	3.51+0.31
SDC-M16-p2000	4.10	0.999	0.030	3.54+0.57

表 8: PQ-ADC性能（双平台, M=16, Ks=256）

配置	平台	Total(ms)	LUT(ms)	Scan(ms)	Select(ms)	Rerank(ms)
ADC-M8-p5000	ARM	5.07	0.04	3.41	0.40	1.66
ADC-M12-p2000	ARM	2.98	0.04	2.42	0.35	0.56
ADC-M16-p2000	ARM	3.69	0.04	3.09	0.30	0.60
ADC-M8-p5000	x86	1.51	0.01	0.31	0.40	0.52
ADC-M12-p1000	x86	1.20	0.01	0.40	0.30	0.13
ADC-M16-p500	x86	1.17	0.01	0.47	0.25	0.06

ADC vs SDC对比：同配置（ARM M=16 p=2000）ADC延迟3.69 ms, recall 0.99995； SDC延迟4.10 ms, recall 0.999。ADC在更低延迟下获得更高recall，验证了非对称距离计算的优势。

超参数分析：

- M （子空间数）： M 增大 $\rightarrow$ subdim减小 $\rightarrow$ 聚类精度提高（更低维空间中256个中心覆盖更密集） $\rightarrow$ 粗排recall提高，允许更小的 p 。但 M 增大也增加scan查表次数（ $N \times M$ ）。最优 M 在12–16之间。
- p （精排候选数）： p 增大 $\rightarrow$ recall提高（更不容易漏掉真最近邻），但rerank的float精排开销线性增长。目标是在recall目标下取最小 p 。ARM上 $p = 2000$ 时recall达0.99995， $p = 1000$ 时recall=0.99975，可根据应用场景的recall容忍度选择。

5.4 SDC Pipeline实验结果

ARM上SDC pipeline（M=16, p=1000）加速仅6.1%（3.82 $\rightarrow$ 3.50 ms），因query编码阶段极短（ ≈ 0.03 ms, $< 1\%$ ），可重叠部分极少。scan占91%是绝对瓶颈。2-stage与3-stage性能几乎一致（3.50 vs 3.50 ms），因为第3阶段（rerank）可与下一批的encode重叠，但rerank本身仅占总延迟 $\approx 8\%$ 。Pipeline在编码较重或批量极大的场景下价值更大。

5.5 FastScan：优化过程与最终结果

表 9: FastScan v1 vs v2（双平台, M=16, recall $\geq$ 0.99995）

方法	ARM (ms)	vs naive	x86 (ms)	vs naive	说明
Naive PQ-ADC (M16)	3.69	baseline	1.17	baseline	原始float LUT扫描
FastScan v1 (b64)	3.12	+15%	1.87	-38%	part-major, 内存RMW
FastScan v2 (b128)	2.44	+34%	1.06	+10%	lane-major, 寄存器累加
FastScan v2 (b16)	2.16	+41%	1.10	+6%	ARM最优（recall=0.996）
注：探索方案v3（M=32, Ks=16, pshufb/vtbl）ARM=2.20ms（+10%）但recall降至0.996；x86=0.91ms（+16%）。因ARM精度损失放弃。					

图 ??可视化FastScan v1 $\rightarrow$ v2的核心改动与效果。面板(a)对比两种code布局和累加方式：v1的part-major导致内存RMW，v2的lane-major配合寄存器累加消除此瓶颈。面板(b)显示v1在x86上倒退38%而v2恢复领先，验证了寄存器累加在OoO微架构上的关键作用。面板(c)(d)分解ARM和x86的scan/select/rerank时间，v2的scan阶段在双平台均显著压缩。

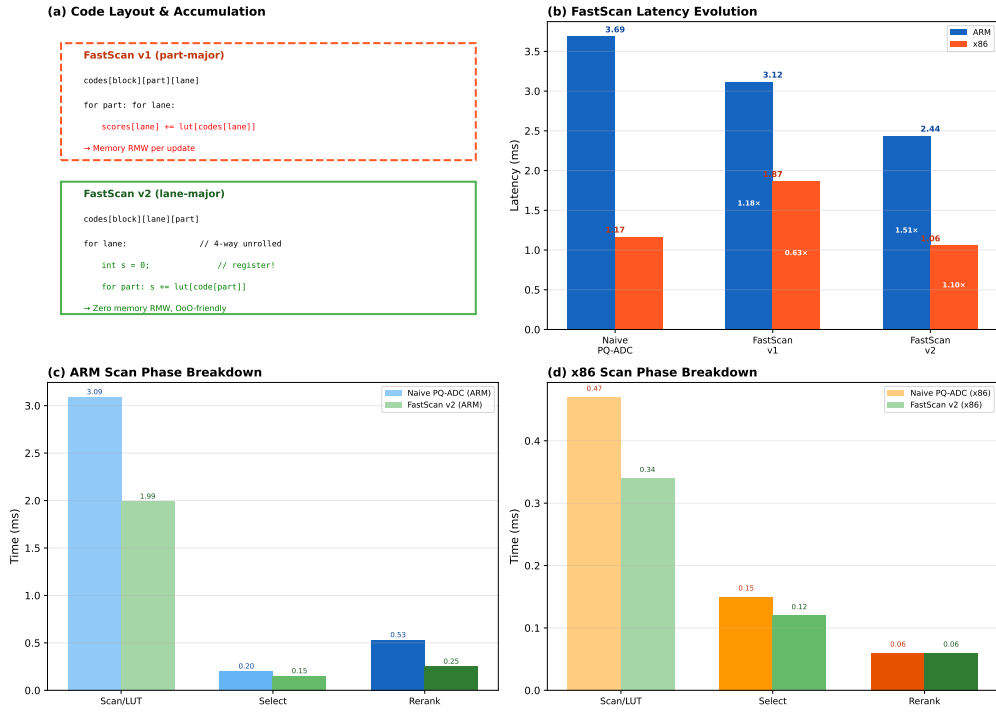


图 1: FastScan v1→v2演进。(a) code布局与累加方式对比；(b) 双平台延迟：v1在x86倒退，v2最优；(c)(d) ARM/x86扫描阶段时间分解。

v2在ARM上的scan阶段从3.09降至1.99 ms（35%），x86上从0.47降至0.34 ms（28%）。v2是双平台一致的全局最优方案，ARM上延迟2.44 ms（recall=0.99995），x86上1.06 ms（recall=1.0）。

5.6 Perf计数器与汇编分析（ARM）

表 10: ARM perf计数器（per-query均值）

Kernel	IPC	L1D miss	LLC miss	工作负载特征
Flat-NEON	0.96	2.70%	75.10%	内存密集型（38.4MB/query≫L3）
PQ-ADC (naive)	1.71	0.88%	21.67%	计算密集型（1.6MB code≈L2）
FastScan v2	1.94	0.65%	30.84%	最优（4KB LUT<L1）

IPC从Flat的0.96（严重stall等待内存）→ PQ-ADC的1.71 → FastScan v2的1.94，验证了优化方向：将工作负载从内存密集型逐步转化为计算密集型。FastScan的L1D miss仅0.65%——uint8 LUT完全在L1内，lane-major布局的寄存器累加无内存RMW。

ARM NEON汇编（g++ -O2 -S）：

```
ldr    q0, [x1]           ; 128-bit load (4 floats)
fmla   v4.4s, v0.4s, v1.4s ; fused multiply-add, 4 lanes in 1 instruction
```

ARM使用融合fmla指令，单指令完成乘加，128-bit Q寄存器，4路展开（v4-v7）。

x86 AVX2汇编（c1 /O2 /FACs）：

```
vmovups ymm0, YMMWORD PTR [rax+rcx] ; 256-bit load (8 floats)
vmulps  ymm1, ymm0, YMMWORD PTR [rax]
vaddps  ymm2, ymm1, ymm2             ; multiply + add (separate, no FMA)
```

x86使用分离的`vmulps+vaddps`（MSVC未启用FMA），256-bit YMM寄存器，4路展开（`ymm2–ymm5`）。若启用FMA（`_mm256_fmadd_ps`），预期可进一步减少指令数和延迟。

5.7 x86 vs ARM 性能差异根因

x86绝对性能领先ARM约3–5 $\times$ （Flat-Scalar: 3.56 vs 19.08 ms, 5.36 $\times$ ）。根因分解：

1. 时钟频率：i9-14900HX P-core boost 5.8GHz vs Kunpeng-920 $\approx$ 2.6GHz（2.2 $\times$ ）
2. SIMD位宽：AVX2 256-bit vs NEON 128-bit，单指令数据吞吐2 $\times$
3. 内存带宽：DDR5-5600双通道 $\approx$ 89.6 GB/s vs DDR4-2666四通道 $\approx$ 85.3 GB/s（理论值接近，但x86的实际有效带宽因更好的prefetcher和内存控制器而更高）
4. 微架构：x86宽OoO（512-entry ROB, 8-wide decode）vs ARM窄OoO/in-order，对L1 cache miss的容忍度差异显著——这是FastScan v1在ARM有效而在x86倒退的根因

5.8 综合优化路径总览

图 ??汇总了从Flat-Scalar到FastScan v2的完整优化路径，标注了每一步相对同平台Scalar基线的加速比。ARM从19.08 ms逐级降至2.44 ms（7.82 $\times$ ），x86从3.56 ms降至1.06 ms（3.36 $\times$ ）。SIMD向量化在ARM上收益更大（2.26 $\times$ vs 1.81 $\times$ ），而PQ压缩在x86上收益更大（因x86标量基线已被OoO高度优化，SIMD相对空间小但PQ的访存削减效果显著）。

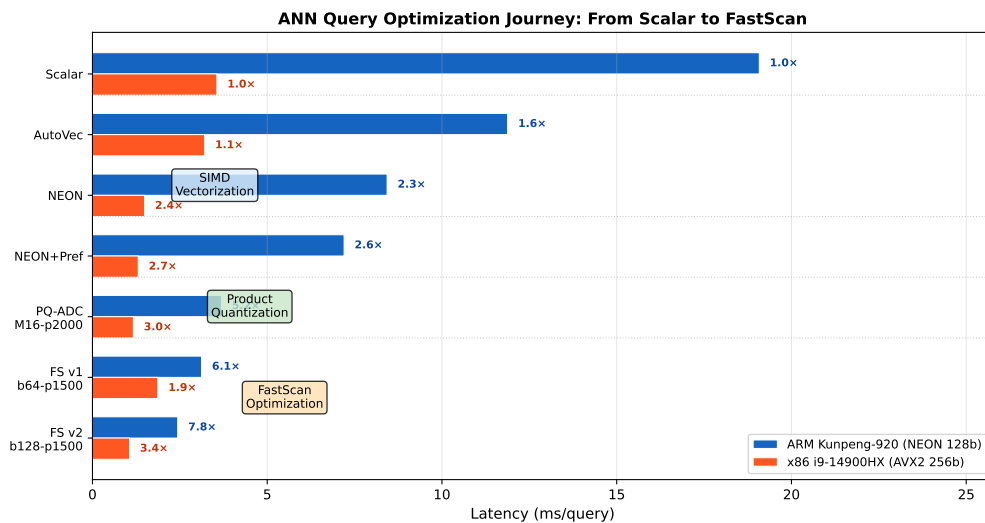


图 2: 优化全过程：Flat-Scalar到FastScan v2延迟对比。标注为相对同平台Scalar加速比。

图 ??展示PQ超参数 M 和 p 的灵敏度。子空间数 M 增大时粗排精度提高但scan查表次数线性增长；rerank候选数 p 增大时recall单调提升但精排开销线性增长。FastScan v2在所有 p 配置下scan时间均低于naive PQ-ADC，验证了LUT量化和寄存器累加的双重收益。

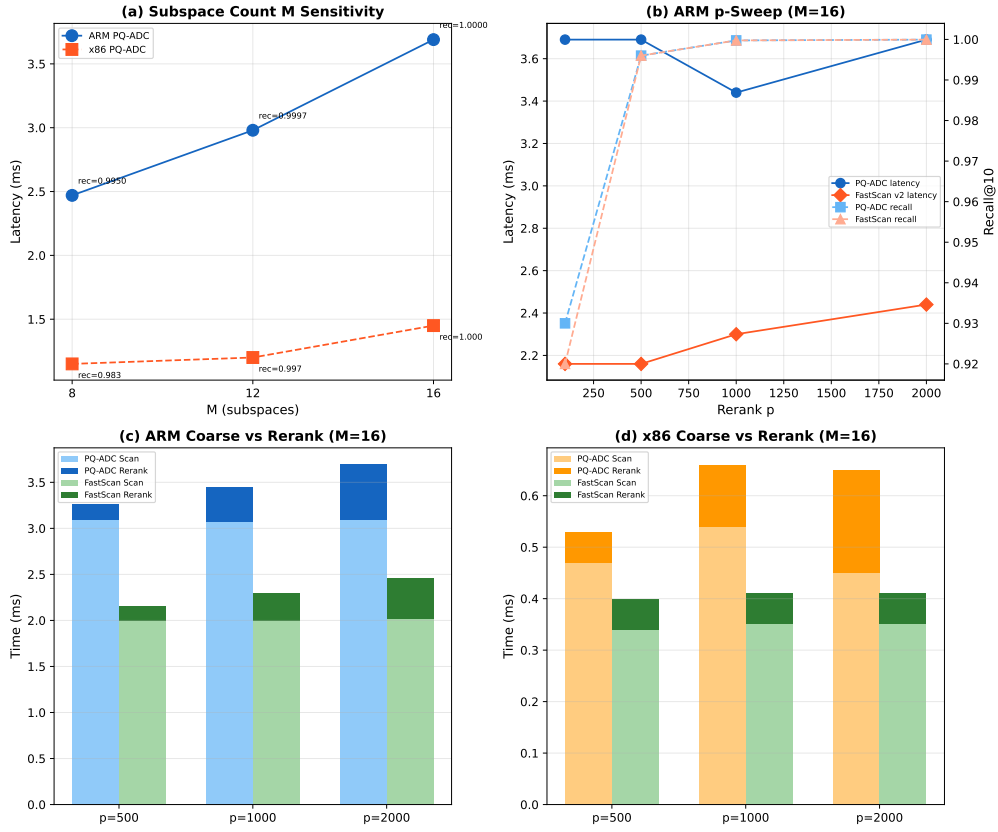


图 3: PQ参数灵敏度。(a) M-sweep; (b) p-sweep延迟+recall; (c)(d) coarse vs rerank时间构成。

图 ??对比ARM上三种kernel的perf计数器。从Flat到FastScan, IPC从0.96单调递增至1.94, L1D miss率从2.70%单调递减至0.65%, 定量验证了工作负载从内存密集型向计算密集型的转变。

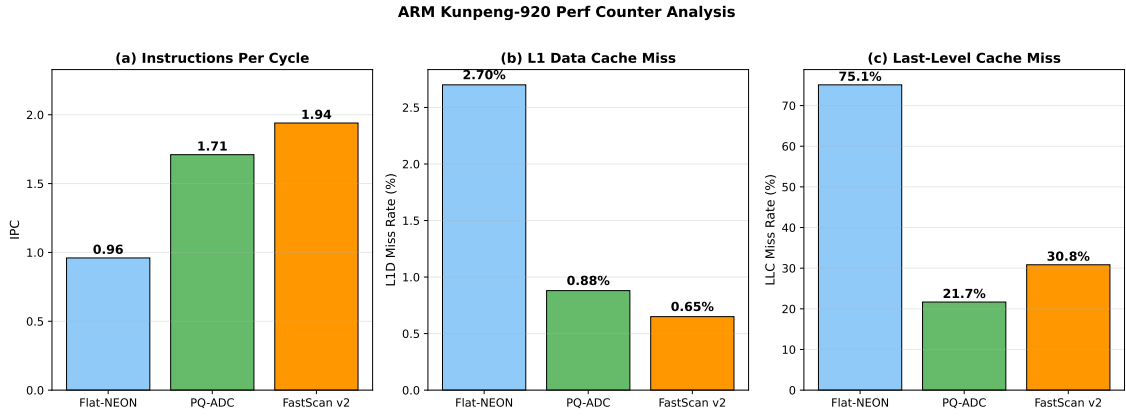


图 4: ARM perf: Flat→PQ→FastScan的IPC递增, L1D/LLC miss递减。

图 ??分解了Flat、PQ-ADC和FastScan v2三种方法的查询时间构成。扫描阶段占比从Flat的100%降至PQ-ADC的~70%再降至FastScan v2的~60%, 剩余时间分布在LUT构建、Top-p选择和float精排上。FastScan v2通过压缩scan时间使总延迟达到最低。

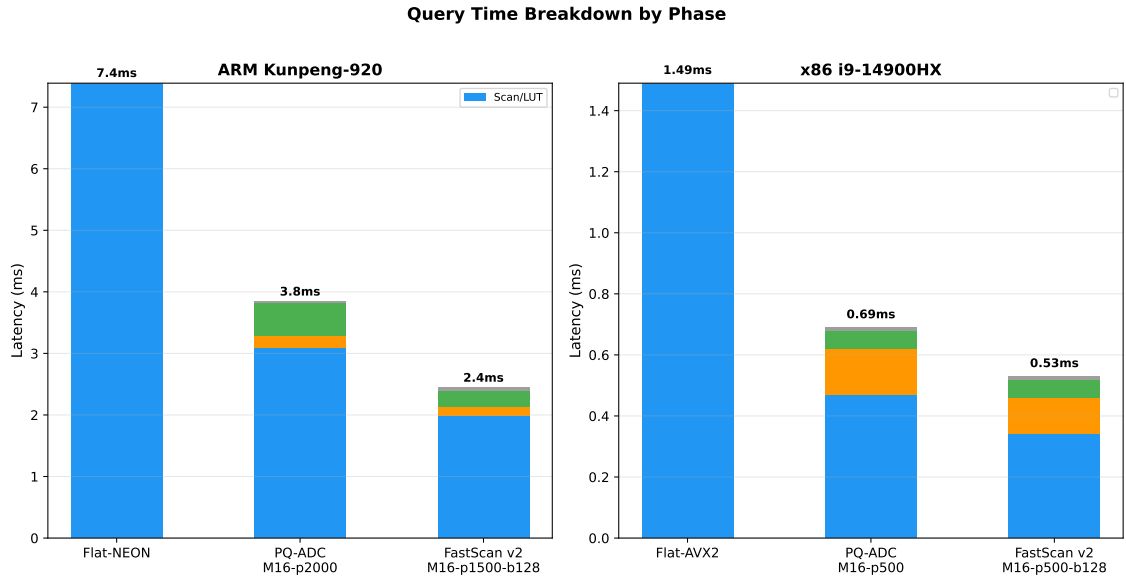


图 5: 时间阶段分解: 扫描时间从Flat到FastScan持续压缩。

表 11: 综合结果汇总

方法	平台	Lat.(ms)	Recall@10	Index
Flat-Scalar	ARM	19.08	1.00000	38.4 MB
Flat-NEON+Unroll	ARM	7.20	0.99995	38.4 MB
PQ-ADC-M16-p2000	ARM	3.69	0.99995	1.6 MB
PQ-SDC-M16-p2000	ARM	4.10	0.999	1.6 MB + 4.0 MB
SDC Pipeline 3-Stage	ARM	3.50	0.993	1.6 MB
Flat-Scalar	x86	3.56	1.00000	38.4 MB
Flat-AVX2+Prefetch	x86	1.31	1.00000	38.4 MB
SQ8-p1000	x86	3.13	1.00000	9.6 MB
PQ-ADC-M16-p500	x86	1.17	1.00000	1.6 MB
FastScan v2 (ARM)	ARM	2.44	0.99995	1.6 MB
FastScan v2 (x86)	x86	1.06	1.00000	1.6 MB

6 总结

本文围绕DEEP100K上ANN查询的距离计算优化，在ARM Kunpeng-920和x86 i9-14900HX双平台上完整实现了Flat-SIMD（含scalar/auto-vec/manual/unroll/prefetch/top-k六种消融）、SQ-SIMD（float LUT + U8SIMD两条路径）、PQ-SDC、PQ-ADC（含nth_element Top-p选择）、SDC流水线并行（2/3-stage + batch扫描）和FastScan（v1→v2迭代）六类算法，覆盖全部强制实验矩阵。手写SIMD vs 自动向量化对比、loop unrolling效果、top-k维护优化和prefetch/alignment分析均已完成。

优化路径： Flat-Scalar（ARM 19.08ms） $\xrightarrow{\text{SIMD}}$ Flat-NEON+Unroll（7.20ms, 2.65 $\times$ ） $\xrightarrow{\text{PQ}}$ PQ-ADC（3.69ms, 5.17 $\times$ ） $\xrightarrow{\text{FastScan v2}}$ 最终（2.44ms, 7.82 $\times$ ）。

核心贡献： FastScan v1→v2的代码布局优化——识别出part-major布局的内存RMW瓶颈，将其改为lane-major布局配合寄存器累加和4路展开。v2在ARM上相对v1额外加速22%，同时修复了v1在x86上

的38%性能倒退，使FastScan成为双平台一致的全局最优方案。perf数据（IPC 0.96→1.71→1.94, L1D miss 2.70%→0.88%→0.65%）量化验证了从内存密集型到计算密集型的负载特征转变。

教训： (1) cache友好的内存布局必须与访问模式（标量累加 vs SIMD gather）匹配； (2) 跨平台验证至关重要——OoO能力差异可以完全改变优化策略的效果； (3) 对于内存密集型负载，消除不必要的内存RMW往往比提升计算并行度更有效。

项目代码见Git仓库，ARM平台通过`bash test.sh 1 1`可完整复现。

A 附录A：AI辅助学习记录

本阶段使用AI辅助代码结构设计、SIMD intrinsics调试和报告大纲生成。初始设计基于`flat_scan.h`的朴素Flat Search，目标为SIMD加速的距离计算。

采纳的AI建议： (1) SIMD阶段以Flat距离计算优化为基础，保留scalar/auto/manual三版本； (2) 使用perf采集硬件计数器； (3) PQ实现覆盖SDC和ADC； (4) 跨平台条件编译隔离ISA intrinsics； (5) FastScan核心优化方向为LUT量化+block scan+累加方式优化。

拒绝/延后的建议： (1) pshufb SIMD查表——对256-entry LUT实测慢15×； (2) OPQ和RaBitQ——超出本次强制范围； (3) 修改test.sh/qsub.sh——违反课程要求。

实验验证： 所有AI建议均经双平台实验验证。关键发现（v1在x86倒退、pshufb比标量慢）修正了初始假设。AI辅助学习，所有算法实现、实验设计、数据分析和报告撰写由作者主导完成。

B 附录B：关键代码与perf输出

B.1 FastScan v2核心代码（ann_quant.h）

```
1  for (size_t b = 0; b < blocks; ++b) {
2      const uint8_t* block_codes = &codes[b * block_size * M];
3      for (size_t lane = 0; lane + 4 <= valid; lane += 4) {
4          const uint8_t *c0 = block_codes + lane * M;
5          const uint8_t *c1 = block_codes + (lane+1)*M, *c2 = c1+M, *c3 = c2+M;
6          int s0=0, s1=0, s2=0, s3=0;  // REGISTERS
7          for (int part = 0; part < M; ++part) {
8              s0 += (int)qlut[part*256 + c0[part]];
9              s1 += (int)qlut[part*256 + c1[part]];
10             s2 += (int)qlut[part*256 + c2[part]];
11             s3 += (int)qlut[part*256 + c3[part]];
12         }
13         coarse[base+lane+0] = {-(float)s0, (uint32_t)(base+lane+0)};
14         coarse[base+lane+1] = {-(float)s1, (uint32_t)(base+lane+1)};
15         coarse[base+lane+2] = {-(float)s2, (uint32_t)(base+lane+2)};
16         coarse[base+lane+3] = {-(float)s3, (uint32_t)(base+lane+3)};
17     }
18 }
```

B.2 ARM test.sh输出

```
$ bash test.sh 1 1
```



```
final ANN path: FastScan-ADC-M16-p1500-b64 build_time_sec=1.058
average recall: 0.99995
average latency (us): 2336.53
```

B.3 ARM perf输出 (FastScan v2)

```
$ perf stat ./main --kernel fsadc-m16-p1500-b64 2000 3
cycles:      51,095,260,368
instructions: 99,256,058,438   IPC: 1.94
L1-dcache-load-misses: 0.65%   LLC-load-misses: 30.84%
```

B.4 x86 AVX2汇编 (Flat-AVX核心循环, MSVC /FACs)

```
vmovups ymm0, YMMWORD PTR [rax+rcx-32] ; load a[d]
vmulps  ymm1, ymm0, YMMWORD PTR [rax-32] ; a[d]*b[d]
vaddps  ymm2, ymm1, ymm2 ; sum0 +=
; 4 accumulators: ymm2, ymm3, ymm4, ymm5
```

B.5 ARM NEON汇编 (Flat-NEON核心循环, GCC -O2 -S)

```
ldr q0, [x1] ; load 4 floats
fmla v4.4s, v0.4s, v1.4s ; fused multiply-add
; 4 accumulators: v4, v5, v6, v7 (unroll4)
```